

An Industrial Oriented Mini Project Report

on

**GEOSCOPE: INTELLIGENT EARTH DATA
PLATFORM**

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

23WH1A05E3 Ms. V. Ishanvi

23WH1A05F0 Ms. V. Sai Sumedha

23WH1A05F2 Ms. P. Praneetha

Under the esteemed guidance of

Ms. Y Divya

Assistant Professor



Department of Computer Science & Engineering

BVRIT HYDERABAD College of Engineering for Women

(NAAC Accredited-A Grade | NBA Accredited B.Tech (EEE, ECE & CSE))

(UGC Autonomous, Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad – 500090

April, 2026



BVRIT HYDERABAD

College of Engineering for Women

(NAAC Accredited-A Grade | NBA Accredited B.Tech. (EEE, ECE & CSE))

(UGC Autonomous, Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad – 500090

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the Industrial Oriented Mini Project entitled “**GEOSCOPE: INTELLIGENT EARTH DATA PLATFORM**” is a bonafide work carried out by **Ms. V. Ishanvi (23WH1A05E3), Ms. V. Sai Sumedha (23WH1A05F0), and Ms. P. Praneetha (23WH1A05F2)** in the partial fulfillment for the award of B.Tech. degree in **Computer Science & Engineering, BVRIT HYDERABAD College of Engineering for Women**, Bachupally, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad, Hyderabad under my guidance and supervision. The results embodied in this Industrial Oriented Mini Project work have not been submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide

Ms. Y Divya

Assistant Professor

Department of CSE

Head of the Department

Dr. Aruna Rao S L

Professor

Department of CSE

External Examiner

DECLARATION

We hereby declare that the work presented in this Industrial Oriented Mini Project entitled **“GEOSCOPE: INTELLIGENT EARTH DATA PLATFORM”** submitted towards completion of Industrial Oriented Mini Project work in III Year II Semester of B.Tech. in CSE at BVRIT HYDERABAD College of Engineering for Women, Hyderabad is an authentic record of our original work carried out under the guidance of **Ms. Y Divya**, Assistant Professor, Department of CSE.

Ms. V. Ishanvi
(23WH1A05E3)

Ms. V. Sai Sumedha
(23WH1A05F0)

Ms. P. Praneetha
(23WH1A05F2)

ACKNOWLEDGMENT

We would like to express our sincere thanks to **Dr. K V N Sunitha, Principal, BVRIT HYDERABAD College of Engineering for Women**, for providing the working facilities in the college.

We are extremely thankful and indebted to our HoD, **Dr. Aruna Rao S L, Professor, Department of CSE, BVRIT HYDERABAD College of Engineering for Women**, for her timely support, valuable suggestions, constant guidance, encouragement, and moral support throughout the project.

We are also deeply grateful to our internal guide, **Ms. Y Divya, Assistant Professor, Department of CSE, BVRIT HYDERABAD College of Engineering for Women**, for her continuous guidance, support, and encouragement during the course of this project.

Finally, we would also like to thank our Industrial Oriented Mini Project Coordinators **Dr. S. Vidyullatha, Assistant Professor and Ms. S. Rama Devi, Associate Professor**, along with all the faculty members and staff of CSE Department who helped us directly or indirectly, parents and friends for their cooperation in completing the project work.

Ms. V. Ishanvi
(23WH1A05E3)

Ms. V. Sai Sumedha
(23WH1A05F0)

Ms. P. Praneetha
(23WH1A05F2)

ABSTRACT

GeoScope is an AI-assisted geospatial dataset discovery platform developed to simplify access to Earth observation data by combining natural-language search, scientific API integration, and interactive visualization in a full-stack web application; unlike traditional catalog browsing workflows that require manual filtering and metadata expertise, GeoScope allows users to submit queries such as “flood in India 2019,” after which a Python Flask backend parses query intent (keyword, year, and region), builds NASA CMR-compatible parameters, retrieves relevant collections in real time from NASA CMR APIs, normalizes heterogeneous metadata into a consistent response format, derives map-ready spatial hints from collection and granule information, and generates structured summaries using Google Gemini with deterministic fallback logic when AI output is unavailable, while a React + Vite frontend presents the complete exploration flow through a 3D-enabled landing experience, map-based result visualization with interactive markers, recommended dataset cards, and a dedicated dataset detail explorer that supports overview, spatial preview, metadata inspection, and raw JSON exploration; the project demonstrates that integrating retrieval pipelines, lightweight query understanding, AI summarization, and user-centered interface design can significantly improve the usability of complex satellite data systems and provides a practical prototype foundation for future enhancements in spatial accuracy, personalization, and production-scale deployment.

Keywords: *Geospatial Data Discovery, NASA CMR, Natural Language Search, AI Summarization, Flask, React, Vite, Satellite Dataset Exploration.*

TABLE OF CONTENTS

ABSTRACT	v
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF TERMS AND ABBREVIATIONS	xi
1 INTRODUCTION	1
1.1 Background	1
1.2 Need for the Project	1
1.3 Problem Statement	1
1.4 Objectives of the Project	2
1.5 Scope of the Project	2
1.6 Expected Contributions	2
2 LITERATURE REVIEW	3
2.1 Literature Survey	3
2.2 Review of Existing Systems	5
2.3 Research Gap	5
2.4 Limitations of Existing Systems	6
3 SYSTEM OVERVIEW AND ARCHITECTURE	7
3.1 Overview of the Proposed System	7
3.2 Operational Flow	7
3.3 Key Features of the System	8
3.4 System Users	9
3.5 Advantages of the Proposed System	9
3.6 System Architecture Overview	9
3.7 Major Components	10

3.8	Backend Architecture	11
3.9	Frontend Architecture	11
3.10	Data Flow	12
3.11	Deployment and Configuration	12
4	METHODOLOGY	13
4.1	Methodology Overview	13
4.2	Query Parsing and Interpretation	14
4.3	Retrieval Pipeline	14
4.4	Scoring and Ranking	14
4.5	Retrieval-Augmented Generation (RAG) and Gemini Integration . . .	15
4.6	Result Presentation	15
5	TOOLS AND TECHNOLOGIES	16
5.1	Software Technologies	16
5.2	Development Environment	18
5.3	Models and AI Components Used	18
5.4	API Utilization	18
5.5	Why This Stack Was Chosen	18
6	SYSTEM IMPLEMENTATION AND RESULTS	19
6.1	Implementation	19
6.1.1	Implementation Overview	19
6.1.2	Backend Implementation	19
6.1.3	Web Application Implementation	20
6.1.4	AI Summary Implementation	22
6.1.5	Backend-Frontend Communication	23
6.2	Results and Analysis	23
6.2.1	Experimental Setup	23
6.2.2	Evaluation Metrics	23
6.2.3	Observed System Performance	23
6.2.4	Qualitative Dataset Analysis	24
6.2.5	AI Summary Analysis	24
6.2.6	Research Assistant Integration	25

6.2.7	Results	25
6.2.8	Limitations of the Results	26
7	CONCLUSION AND FUTURE SCOPE	27
7.1	Conclusion	27
7.2	Future Scope	27
	REFERENCES	29
	Appendices	
	Appendix A Core Code Snippets	31

LIST OF FIGURES

3.1	Overall workflow of GeoScope from user query to dataset discovery and visualization	8
3.2	High-level architecture of the GeoScope platform	10
4.1	Methodology adopted in GeoScope for query interpretation, hybrid retrieval, ranking, and AI-powered explanation	13
6.1	Search interface for entering natural-language Earth data queries and Map pinpointing locations	21
6.2	Landing page with 3D Earth visualization and exploration entry	21
6.3	Dataset recommendation cards with summary context and quick actions	22
6.4	Detailed dataset explorer with metadata tabs and raw payload view . . .	22
6.5	The Research Assistant extracting specific volcano data from a dataset based on a user query	25

LIST OF TABLES

2.1	Comparison of Earth Data Discovery Systems	5
5.1	Software Stack Used in GeoScope	16
6.1	Representative System Performance	24

LIST OF TERMS AND ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25 (Search Algorithm)
CMR	Common Metadata Repository (NASA)
CORS	Cross-Origin Resource Sharing
FAISS	Facebook AI Similarity Search
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
ML	Machine Learning
NASA	National Aeronautics and Space Administration
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SPA	Single Page Application
SVD	Singular Value Decomposition
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
UX	User Experience
WSGI	Web Server Gateway Interface

CHAPTER 1

INTRODUCTION

1.1 Background

GeoScope is a full-stack Earth data discovery application built around NASA’s Common Metadata Repository (CMR) search. The project enables users to enter natural-language queries such as “flood in India 2019” and translates them into NASA CMR collection searches. The results are presented in a visual user interface with a landing page, interactive map, dataset detail views, and AI-generated summary bullets. GeoScope combines live NASA CMR search, a local ingested corpus, hybrid retrieval (BM25 and vector similarity), and Gemini-powered explanations to provide a rich discovery experience for Earth science data.

1.2 Need for the Project

Traditional Earth data search tools often require users to know specific dataset names or technical keywords, making discovery difficult for non-experts. There is a need for a system that accepts natural-language queries, understands temporal and regional context, and presents results in an accessible, visual format. GeoScope addresses this gap by supporting flexible queries, integrating multiple retrieval strategies, and providing grounded AI explanations to help users interpret and compare datasets.

1.3 Problem Statement

Existing Earth data portals primarily focus on keyword-based search and often lack intuitive interfaces for exploring datasets by event, region, or time. Users may struggle to find relevant data due to limited query understanding, lack of hybrid retrieval, or insufficient result explanations. There is a need for a discovery platform that combines advanced search, semantic understanding, and user-friendly visualization to make

NASA Earth science data more accessible.

1.4 Objectives of the Project

The main objectives of GeoScope are as follows:

- To design a search framework that supports natural-language queries and interprets temporal and regional filters.
- To implement hybrid retrieval using both lexical (BM25) and vector similarity for improved dataset ranking.
- To integrate live NASA CMR search with a local indexed corpus for flexible and robust discovery.
- To provide AI-generated summaries and grounded explanations using Gemini for enhanced result interpretation.
- To build a React-based frontend with map visualization, dataset detail pages, and assistant features.

1.5 Scope of the Project

GeoScope currently focuses on NASA CMR collection discovery, supporting queries with year and region extraction, and presenting results on an interactive map. The backend is implemented in Flask, with hybrid retrieval and Gemini integration. The frontend is a React single-page application using Vite and Leaflet for visualization. The system does not include direct granule download, user authentication, or large-scale offline indexing. Future extensions may add support for additional data sources, user accounts, and more advanced analytics.

1.6 Expected Contributions

GeoScope makes the following contributions:

- It reframes Earth data search as a multi-strategy discovery problem, blending live and local retrieval.
- It demonstrates hybrid ranking using both BM25 and vector similarity, with metadata and filter-aware scoring.
- It integrates Gemini-powered explanations and summaries to make results more interpretable.
- It provides a map-centric frontend for exploring and comparing NASA datasets.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Survey

Earth science data discovery has traditionally relied on keyword-based search systems and structured metadata filtering. Most existing platforms require users to input exact dataset names, parameters, or domain-specific terminology. As discussed in [1], this approach creates a significant usability barrier, particularly for non-expert users unfamiliar with technical metadata schemas. Systems like NASA Earthdata Search illustrate this limitation, where effective data retrieval often depends on prior knowledge rather than intuitive exploration.

To overcome these challenges, recent research has focused on semantic and natural-language-driven search techniques. Unlike traditional keyword matching, these approaches aim to capture user intent and contextual meaning. Dense retrieval models, which use neural embeddings, have shown strong performance in this area. For example, [2] highlights how large language model (LLM)-augmented data can improve retrieval quality in dataset search systems. Embedding-based methods, such as sentence-transformers combined with FAISS, enable scalable semantic similarity search, allowing systems to retrieve relevant datasets even when exact keywords are not present.

Hybrid retrieval methods have further improved performance by combining lexical techniques like BM25 with semantic vector search. This combination balances precision and contextual relevance, leading to more accurate results. Additionally, context-aware retrieval models incorporate spatial, temporal, and domain-specific signals into the ranking process [3]. This is particularly important in geospatial domains, where datasets are inherently linked to location and time, and relevance depends on more than just textual similarity.

Metadata quality plays a critical role in improving search effectiveness. Research such as [4] shows that relying solely on keywords is insufficient, and that richer metadata fields—including descriptions, spatial coverage, and temporal attributes—significantly enhance retrieval performance. Furthermore, knowledge graph-based approaches have been explored to improve dataset relationships and discoverability. The work in [5] demonstrates how linking datasets with related publications and concepts can create a more connected and intuitive search experience.

More recently, retrieval-augmented generation (RAG) frameworks have been introduced to enhance dataset exploration. These systems combine retrieval mechanisms with generative models to provide context-aware summaries and explanations. For instance, [6] explores metadata-driven RAG for improved data exploration, while [7] emphasizes the importance of temporal awareness in retrieval systems. Such approaches are especially useful in Earth science applications, where understanding time-based patterns is essential.

AI-powered assistants based on large language models are increasingly being integrated into data discovery platforms. These systems can generate summaries, extract key information, and provide user-friendly explanations of complex datasets. By bridging the gap between technical metadata and user intent, they significantly improve accessibility and usability.

Overall, the transition from keyword-based systems to hybrid, context-aware, and AI-assisted retrieval methods reflects a shift toward more user-centric and intelligent data discovery. While existing research has made substantial progress, challenges remain in integrating semantic understanding, metadata richness, and interactive visualization into a unified system. GeoScope builds upon these advancements by combining natural-language query processing, hybrid retrieval techniques, and interactive geospatial visualization to deliver a more intuitive and effective dataset discovery experience.

2.2 Review of Existing Systems

A comparison of existing Earth data discovery systems reveals their strengths and limitations.

Table 2.1 Comparison of Earth Data Discovery Systems

System	Keyword Search	Semantic Search	AI Summaries	Remarks
NASA Earthdata Search	Yes	No	No	Comprehensive, but requires technical knowledge
Google Dataset Search	Yes	Partial	No	Broad coverage, limited semantic context
Pangeo Forge	Yes	No	No	Focused on cloud-ready data, technical interface
Research prototypes	Partial	Partial	Partial	Often limited to specific domains or datasets
GeoScope	Yes	Yes	Yes	Combines hybrid retrieval, AI explanations, and map-centric UI

Most mainstream systems provide powerful search capabilities but lack natural-language understanding, hybrid ranking, or AI-powered explanations. Users often need to interpret technical metadata themselves.

2.3 Research Gap

The literature and existing systems show progress in isolated areas such as semantic search, metadata quality, and AI summarization. However, there is a gap in integrating these advances into a unified, user-friendly platform that:

- accepts natural-language queries,
- interprets temporal and regional context,
- combines live and local retrieval strategies,
- provides AI-generated summaries and explanations,
- and presents results in an interactive, map-based interface.

GeoScope addresses this gap by integrating hybrid retrieval, Gemini-powered explanations, and a modern frontend for Earth data discovery.

2.4 Limitations of Existing Systems

Key limitations observed in current systems include:

- **Technical barriers:** requiring users to know dataset names or metadata fields.
- **Lack of semantic understanding:** limited ability to interpret user intent in natural language.
- **No hybrid ranking:** reliance on either keyword or vector search, not both.
- **Insufficient explanations:** lack of AI-generated summaries or grounded answers.
- **Limited visualization:** results often presented as lists, not interactive maps[3].

CHAPTER 3

SYSTEM OVERVIEW AND ARCHITECTURE

3.1 Overview of the Proposed System

GeoScope is an end-to-end Earth data discovery platform composed of a React-based frontend, a Flask backend, external data services (NASA CMR, Google Gemini), and a hybrid retrieval and ranking pipeline. The user enters a natural-language query (e.g., event, region, year) in the web interface. The backend parses the query, determines temporal and regional filters, and decides whether to use live NASA CMR search, the local indexed corpus, or a hybrid of both. Retrieved datasets are ranked using lexical (BM25), vector similarity, metadata quality, and filter-aware scoring. Gemini is used for AI-generated summaries and grounded explanations. The frontend displays results on an interactive map, with dataset detail pages and assistant features.

3.2 Operational Flow

The operational flow of GeoScope can be summarized in the following steps:

1. The user enters a natural-language query in the web application.
2. The frontend sends the request to the Flask backend API.
3. The backend parses the query for keywords, year, and region.
4. Search parameters are constructed for NASA CMR and/or the local corpus.
5. The backend performs hybrid retrieval using BM25 and vector similarity.
6. Metadata quality and filter matches are used to refine ranking.
7. Gemini generates summary bullets and explanations for the results.
8. The backend returns structured results, summaries, and metadata.
9. The frontend displays the results as map markers, cards, and detail pages.
10. The user can explore datasets, view explanations, and compare results.

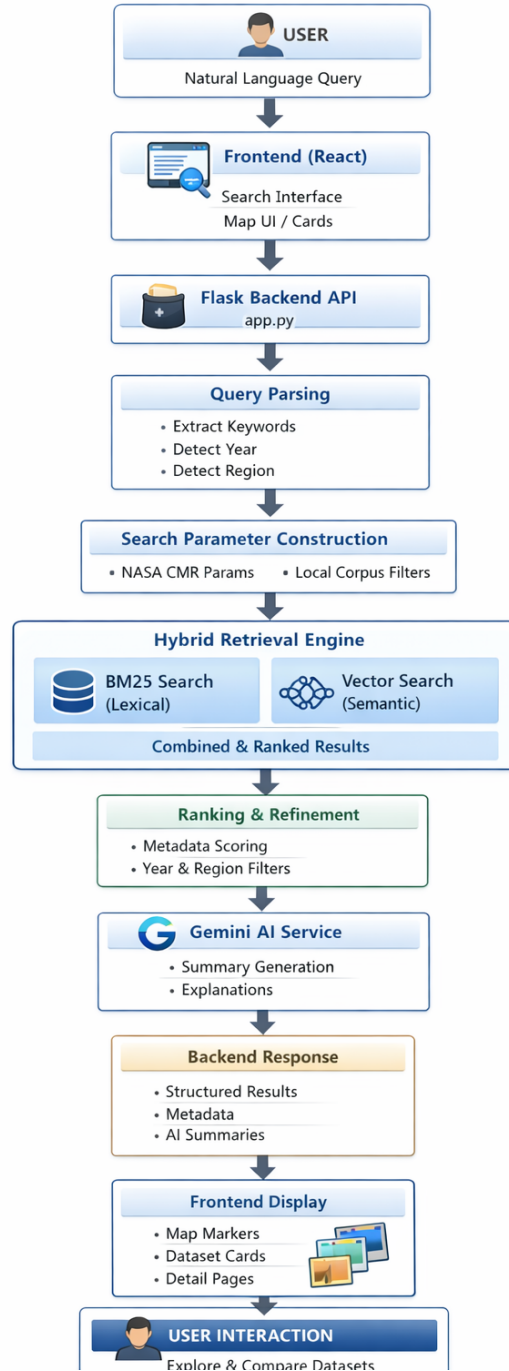


Fig. 3.1 Overall workflow of GeoScope from user query to dataset discovery and visualization

3.3 Key Features of the System

The proposed system includes the following major features:

- Natural-language query support with temporal and regional parsing.
- Hybrid retrieval using both BM25 lexical search and vector similarity.
- Integration of live NASA CMR search and local indexed corpus.
- AI-generated summaries and explanations using Gemini.

- Interactive map visualization with dataset markers.
- Dataset detail pages with rich metadata and raw JSON views.
- Assistant features for grounded, citation-friendly answers.
- Comparison of datasets by relevance, metadata quality, and filters.

3.4 System Users

GeoScope is designed for the following user groups:

- **Researchers and scientists:** users seeking relevant Earth science datasets for analysis.
- **Students and educators:** users exploring geospatial data for learning and teaching.
- **Policy makers and analysts:** users needing accessible, interpretable data for decision-making.
- **General public and enthusiasts:** users interested in discovering Earth observation data.

3.5 Advantages of the Proposed System

Compared with conventional Earth data portals, GeoScope offers the following advantages:

- It supports flexible, human-friendly queries and interprets user intent.
- It blends multiple retrieval strategies for robust and relevant results.
- It provides AI-powered summaries and explanations for better understanding.
- It presents results in a modern, map-centric, and interactive interface.
- It is modular and extensible, allowing future integration of new data sources and features.

3.6 System Architecture Overview

GeoScope is architected as a modular, full-stack Earth data discovery platform. It consists of a React + Vite frontend, a Flask backend API, and integrations with NASA CMR, Google Gemini, and local data storage. The architecture is designed to support hybrid retrieval, AI-powered explanations, and interactive visualization, making NASA Earth science data accessible to a broad range of users.

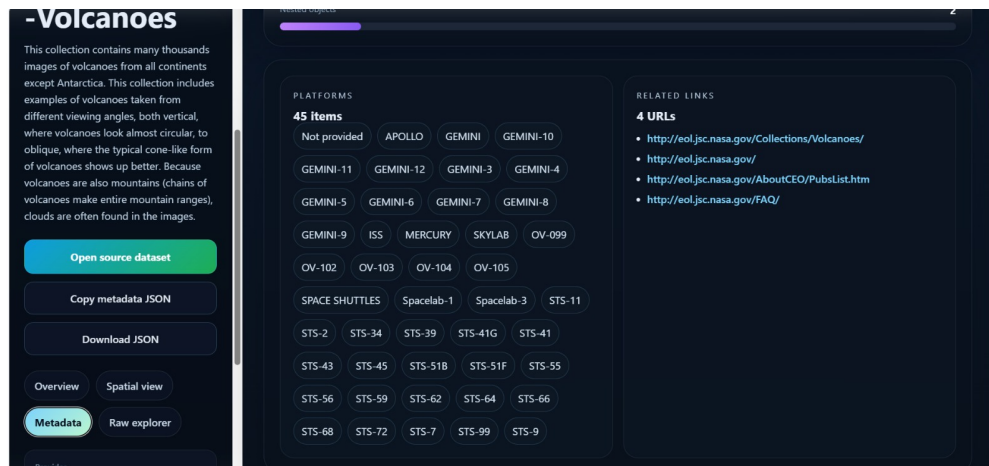


Fig. 3.2 High-level architecture of the GeoScope platform

3.7 Major Components

The main components of GeoScope are:

- **Frontend (React + Vite):**
 - Provides a single-page application (SPA) interface.
 - Implements routes for landing page, map-based search, and dataset detail views.
 - Renders search results as interactive map markers and cards.
 - Handles user input, query submission, and navigation.
 - Consumes the backend API for all data and explanations.
- **Backend (Flask API):**
 - Orchestrates query parsing, search, retrieval, and ranking.
 - Integrates with NASA CMR for live dataset discovery.
 - Manages a local ingested corpus in JSONL and SQLite formats.
 - Implements hybrid retrieval using BM25 and vector similarity (FAISS).
 - Applies metadata quality and filter-aware scoring.
 - Calls Gemini for keyword extraction, summaries, and grounded explanations.
 - Exposes REST endpoints for search, dataset detail, and assistant features.
- **External Integrations:**
 - **NASA CMR:** Live search and metadata retrieval for Earth science datasets.
 - **Google Gemini:** AI-powered keyword extraction, summaries, and explanations.

- **Esri Imagery:** Map tiles for frontend visualization.
- **Local Data Storage:**
 - Stores ingested NASA CMR metadata as JSONL and SQLite for fast local search.
 - Persists FAISS vector indexes and BM25 artifacts for hybrid retrieval.

3.8 Backend Architecture

The backend is organized into several modules:

- **app.py:** Entry point, API routes, search orchestration, and CORS setup.
- **config.py:** Central configuration for endpoints, weights, and runtime options.
- **services/**
 - **cmr_client.py:** NASA CMR HTTP client, collection normalization, coordinate extraction.
 - **query_parser.py:** Natural-language parsing for year and region filters.
 - **metadata_store.py:** Corpus ingestion, normalization, and storage.
 - **lexical_search.py:** BM25-based search over normalized records.
 - **vector_search.py:** FAISS-backed semantic search with sentence-transformers.
 - **ranker.py:** Combines lexical, vector, and metadata scores.
 - **retriever.py:** High-level hybrid retrieval and query suggestion.
 - **gemini_service.py:** Gemini REST calls for keywords and summaries.
 - **explainer.py:** Dataset explanations for different audiences.
 - **rag_service.py:** Grounded assistant answers using local datasets.
- **prompts/**
 - Prompt templates for Gemini-powered explanations and assistant responses.

3.9 Frontend Architecture

The frontend is a React SPA with the following structure:

- **App.jsx:** Top-level router and landing page.
- **MapPage.jsx:** Main search and map interface.
- **DatasetPage.jsx:** Rich dataset detail and metadata explorer.
- **Reusable components:** Cards, overlays, and map widgets.
- **State management:** Local state for search, results, selection, and summaries.

- **API integration:** Uses Axios to communicate with the Flask backend.
- **Visualization:** Uses Leaflet and Esri tiles for map rendering.

3.10 Data Flow

1. User submits a query via the frontend.
2. Frontend sends the query to the backend API.
3. Backend parses the query, determines filters, and selects retrieval mode.
4. Hybrid retrieval is performed using BM25 and vector similarity.
5. Results are ranked, summarized, and explained using Gemini.
6. Structured results are returned to the frontend.
7. Frontend displays results on the map and in detail views.

3.11 Deployment and Configuration

- **Backend:** Deployed as a Python web service (e.g., via Render), requires NASA CMR access and Gemini API key.
- **Frontend:** Deployed as a static site (e.g., Vercel, Netlify), configured to point to the backend API.
- **Environment:** Uses .env files for API keys and configuration.
- **Artifacts:** Local indexes and metadata are stored outside version control.

CHAPTER 4

METHODOLOGY

4.1 Methodology Overview

The methodology of GeoScope integrates natural-language query parsing, hybrid retrieval (BM25 and vector similarity), metadata normalization, advanced ranking, and AI-powered explanation using Gemini. The system is designed to support flexible user queries, robust dataset discovery, and interpretable results through a modular backend and interactive frontend.

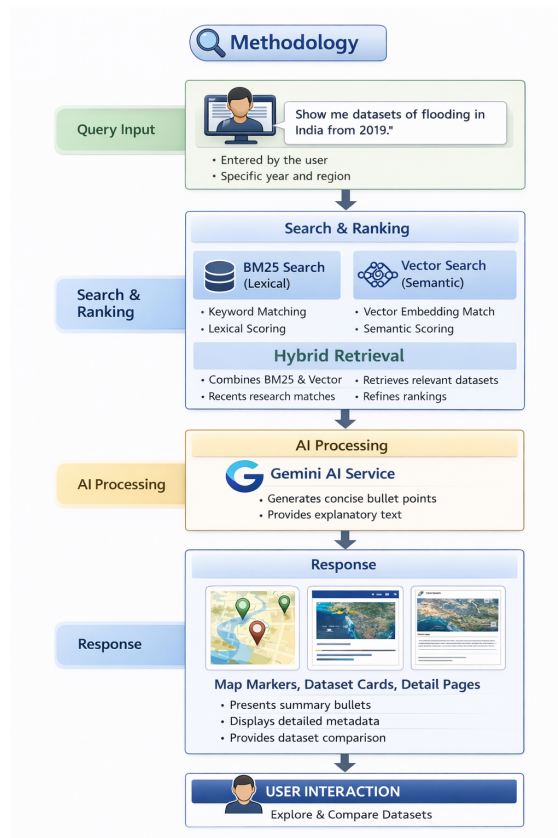


Fig. 4.1 Methodology adopted in GeoScope for query interpretation, hybrid retrieval, ranking, and AI-powered explanation

4.2 Query Parsing and Interpretation

- User queries are parsed by the backend to extract keywords, temporal filters (years), and regional constraints (named bounding boxes).
- The `query_parser.py` module removes year tokens and region names from the search text, converting them into NASA CMR-compatible parameters.
- Supported regions include `india`, `global`, `california`, `europa`, and `indonesia`.
- Example: “flood in India 2019” → keyword: `flood`, year: `2019`, region: `india`.

4.3 Retrieval Pipeline

1. **Mode Selection:** The backend determines whether to use live NASA CMR search, the local indexed corpus, or a hybrid of both, based on corpus availability and query fit.
2. **Lexical Search:** The `lexical_search.py` module performs BM25-based search over normalized records, weighting fields such as title and short name more heavily.
3. **Vector Search:** The `vector_search.py` module uses sentence-transformer embeddings and FAISS for semantic similarity. If embedding fails, it falls back to TF-IDF and SVD.
4. **Hybrid Ranking:** The `ranker.py` module combines lexical and vector results, applies metadata quality and filter-aware scoring, and produces a final ranked list.
5. **Fallbacks:** If the local corpus is empty or matches are weak, the system falls back to live CMR search and may merge results.

4.4 Scoring and Ranking

- **BM25 Score:** Measures lexical relevance between query and dataset metadata.
- **Vector Similarity:** Measures semantic similarity using embeddings.
- **Metadata Quality:** Rewards datasets with rich summaries, spatial metadata, and science keywords.
- **Filter Match:** Adds score for year and region overlap.
- **Hybrid Score:** Weighted blend of all the above, with weights defined in `config.py[2]`.

- **Explanation:** The system records human-readable match reasons for transparency.

4.5 Retrieval-Augmented Generation (RAG) and Gemini Integration

- GeoScope uses Gemini for keyword extraction, dataset summaries, and grounded explanations.
- The `gemini_service.py` module sends prompt templates (from `prompts/`) and retrieved dataset metadata to Gemini via REST API.
- The RAG pipeline ensures that all AI-generated explanations are grounded in actual retrieved datasets, avoiding hallucination.[7]
- If Gemini is unavailable, deterministic fallback summaries are generated.
- The assistant and comparison features use grounded prompts to ensure explanations are based on actual retrieved data.

4.6 Result Presentation

- The frontend receives structured results, including ranked datasets, summary bullets, and metadata.
- Results are visualized as map markers (using Leaflet and Esri tiles), cards, and detail pages.
- Users can explore dataset details, view raw metadata, and access AI-generated explanations.

CHAPTER 5

TOOLS AND TECHNOLOGIES

5.1 Software Technologies

GeoScope uses a full-stack combination of web, backend, API integration, and AI-assisted processing technologies. The selected tools were chosen to keep the prototype practical, interactive, and suitable for academic as well as demonstration use cases.

Table 5.1 Software Stack Used in GeoScope

S.No	Technology	Category	Purpose
1	Python 3.x	Programming Language	Backend logic, API orchestration, query parsing, metadata processing
2	Flask 3.0.0	Web Framework	REST endpoints for search, dataset details, health checks, chat functionality
3	Flask-CORS 4.0.0	Backend Utility	Cross-origin resource sharing between frontend and backend
4	Requests 2.32.3	HTTP Library	Backend HTTP calls to NASA CMR and external APIs
5	python-dotenv 1.0.1	Configuration	Environment variable management and secure API key handling
6	google-generativeai	AI Library	Gemini integration for keyword extraction and result summarization
7	Gunicorn 22.0.0	WSGI Server	Production deployment on cloud platforms like Render

Table 5.1 Software Stack Used in GeoScope (Continued)

S.No	Technology	Category	Purpose
8	React.js	Frontend Framework	Single-page application for search and dataset exploration
9	React-Router-DOM	Routing	Client-side routing for landing, map, and dataset detail pages
10	Vite	Frontend Build Tool	Fast development server and optimized production build pipeline
11	Axios	HTTP Client	Frontend-to-backend API communication with structured requests
12	Leaflet	Mapping Library	Interactive base maps and spatial visualization layers
13	React-Leaflet	React Integration	Leaflet maps as React components with event handling
14	React-Icons	Icon Library	UI icons for buttons, navigation, and interactive controls
15	React-Typed	Animation	Animated typing text effects on landing page
16	React-Tsparticles	Particle System	Optional visual effects and animations
17	Chart.js	Data Visualization	Charts for optional dataset analytics and visualization
18	React-Chartjs-2	React Wrapper	Integration of Chart.js with React components
19	ESLint	Code Linter	Code quality and consistency enforcement

5.2 Development Environment

The backend is developed using Python with `requirements.txt` for dependency management. The frontend uses Node.js with npm for package management and Vite for build orchestration. During local development, the frontend runs on the Vite dev server (default `localhost:5173`) and the backend runs on Flask (default `localhost:5001`). The frontend communicates with the backend via `VITE_API_BASE` environment variable. For production, the backend deploys on Render using Gunicorn, and the frontend is built as a static SPA for Vercel deployment.

5.3 Models and AI Components Used

GeoScope uses a hybrid rule-based retrieval with AI assistance rather than pre-trained ML models:

- **Rule-based query parsing:** extracts event keyword, year, and geographic region from user input
- **NASA CMR retrieval:** live API calls fetch candidate collections with complete metadata
- **Gemini AI service:** keyword refinement and structured JSON summary generation[6]
- **Fallback logic:** deterministic summary generation when AI is unavailable

5.4 API Utilization

External APIs are central to system functionality:

- **NASA CMR Collections API:** retrieves Earth observation datasets with metadata and identifiers
- **NASA CMR Granules API:** refines spatial coordinates from granule-level metadata[5]
- **Google Gemini API:** generates user-readable and technical summary points

5.5 Why This Stack Was Chosen

The stack balances prototyping speed with production capability. Flask provides lightweight APIs, React & Vite deliver a responsive frontend, and Leaflet enables spatial visualization. NASA CMR ensures authoritative data access, while Gemini adds intelligent summarization. This setup is practical for academic demonstration yet scalable for real-world deployment.

CHAPTER 6

SYSTEM IMPLEMENTATION AND RESULTS

6.1 Implementation

6.1.1 Implementation Overview

The implementation of GeoScope is divided into two major parts: the Python Flask backend and the React-Vite web application. The backend performs query interpretation, NASA dataset retrieval, metadata normalization, and AI-based summarization, while the frontend provides the search, map visualization, and dataset exploration interface.

6.1.2 Backend Implementation

6.1.2.1 REST Endpoints

The backend exposes major endpoints:

- `/api/health` for checking server availability,
- `/api/search` for natural-language query processing and dataset retrieval,
- `/api/dataset/<dataset_id>` for detailed dataset metadata,
- `/search` for alternate Gemini-assisted search,
- `/chat` for assistant-style dataset-aware responses.

6.1.2.2 Service Initialization

At startup, the backend initializes Flask, loads environment variables, enables CORS, and configures external integrations. Gemini setup is initialized through the API key, and NASA CMR endpoint constants are prepared for runtime dataset calls.

6.1.2.3 Query Parsing and Feature Extraction Logic

The backend parses free-text user input and extracts structured signals such as keyword, year, and region. Helper logic maps these signals into CMR-compatible parameters, including temporal range and optional bounding-box constraints, which improves relevance of returned collections.

6.1.2.4 Retrieval and Recommendation Pipeline

When the user submits a query, the backend:

1. parses keyword, time, and regional intent,
2. builds NASA CMR search parameters,
3. retrieves candidate collections from CMR,
4. normalizes metadata and link fields,
5. derives representative coordinates for mapping,
6. generates user-facing summary bullets, and
7. returns a structured JSON response to the frontend.

6.1.3 Web Application Implementation

6.1.3.1 Search Interface

The web app provides a natural-language query input for Earth data search (e.g., "volcano"). The interface is designed to make scientific dataset discovery accessible without requiring technical catalog knowledge.

6.1.3.2 Map Rendering

An interactive Leaflet map is used to display dataset markers. Popups provide quick metadata context and direct access to source links and detail navigation, enabling spatial-first dataset inspection.



Fig. 6.1 Search interface for entering natural-language Earth data queries and Map pinpointing locations

6.1.3.3 Landing Experience

The app starts with a visual landing page containing a 3D Earth background and guided prompt text. A clear call-to-action directs users into the map-based exploration workflow.



Fig. 6.2 Landing page with 3D Earth visualization and exploration entry

6.1.3.4 Result Cards and Dataset Navigation

Search results are also shown as recommendation cards with title, summary context, and key metadata. Selecting a result transitions the user to a dedicated dataset detail page for deeper exploration.

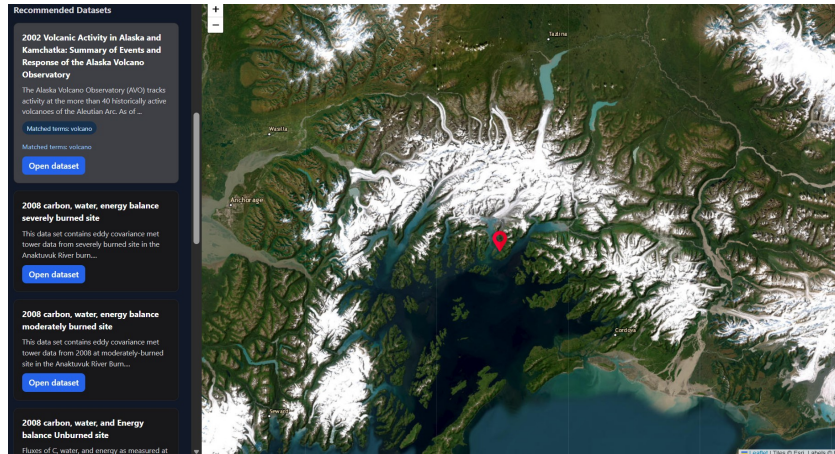


Fig. 6.3 Dataset recommendation cards with summary context and quick actions

6.1.3.5 Dataset Detail View

The detail page presents richer information including overview, spatial footprint, normalized metadata fields, related links, and raw JSON. This supports both general users and technical users who need direct metadata inspection.

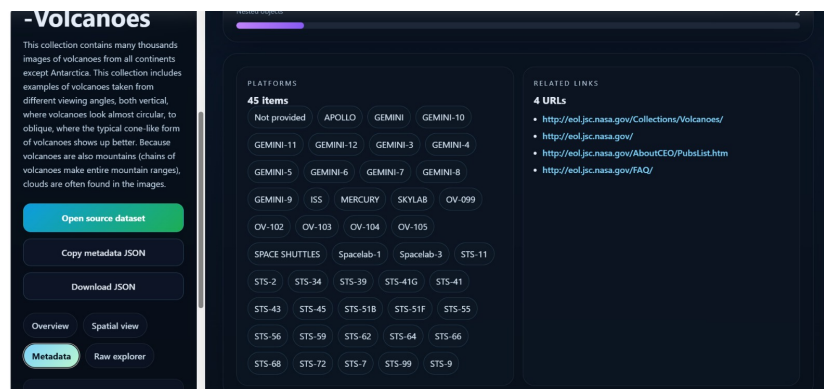


Fig. 6.4 Detailed dataset explorer with metadata tabs and raw payload view

6.1.4 AI Summary Implementation

GeoScope generates two summary streams for returned results: layman-friendly explanation points and satellite-data-focused insights. If AI output is unavailable, a deterministic fallback summary is generated to keep the interface informative and robust.

6.1.5 Backend-Frontend Communication

The web frontend communicates with the backend through a configurable API base URL (`VITE_API_BASE`). Responses include parsed query context, normalized dataset metadata, summary bullets, and coordinates for map rendering. Dataset detail requests fetch enriched metadata for deep inspection views.

6.2 Results and Analysis

6.2.1 Experimental Setup

The system was evaluated through an end-to-end integration of the frontend, backend, and external data sources. User queries were processed using a natural-language parsing pipeline, followed by keyword extraction and refinement using AI models. The backend then interacted with NASA CMR APIs to retrieve relevant datasets, which were processed and sent to the frontend for visualization on an interactive map interface.

6.2.2 Evaluation Metrics

The performance of the system was evaluated based on qualitative and system-level metrics rather than traditional numerical accuracy measures.

- **Relevance of Results** measures how accurately the retrieved datasets match the user's natural-language query.
- **Response Time** evaluates the time taken to process a query and display results.
- **Summary Quality** measures the clarity and usefulness of AI-generated dataset explanations.
- **User Experience** evaluates the ease of navigation and interpretability of the interface.

6.2.3 Observed System Performance

Based on the current implementation, the system demonstrates effective performance in retrieving and presenting relevant datasets. For typical queries, the system:

- successfully extracts key parameters such as keywords, region, and time,
- retrieves multiple relevant datasets from NASA repositories,
- generates meaningful summaries that simplify technical descriptions,

- displays dataset locations interactively on the map interface.

Response times are generally within a few seconds depending on API latency, and the results remain consistent across repeated queries with similar inputs.

Table 6.1 Representative System Performance

Metric	Observation
Query Processing	Accurate keyword and parameter extraction
Dataset Retrieval	Relevant datasets fetched from NASA CMR
Summary Generation	Clear and simplified explanations
Response Time	Typically a few seconds

6.2.4 Qualitative Dataset Analysis

In practical usage, the system enables users to explore datasets more intuitively compared to traditional search interfaces. The results typically exhibit:

- datasets closely aligned with the user’s query intent,
- improved understanding through AI-generated summaries,
- spatial awareness via map-based visualization,
- easy navigation between multiple dataset options.

Users can quickly identify relevant datasets and explore their details without needing deep technical knowledge of satellite data.

6.2.5 AI Summary Analysis

The AI-generated summaries play a key role in improving accessibility. Each dataset is accompanied by:

- a layman-friendly explanation of the dataset purpose,
- insights into the type of satellite data used,
- contextual understanding of how the dataset can be applied.

This significantly reduces the complexity of interpreting raw metadata from NASA sources.

6.2.6 Research Assistant Integration

To further enhance user experience and data accessibility, GeoScope features an integrated "Research Assistant." Rather than simply returning links to datasets, the assistant actively parses the retrieved dataset metadata to answer specific user questions.

As shown in Figure 6.6, when a user queries "active volcanos in alaska," the Research Assistant isolates the highly relevant dataset ("2002 Volcanic Activity in Alaska and Kamchatka") and extracts the exact data points required. It provides a clean, bulleted list of the specific volcanic centers mentioned in the dataset (e.g., Wrangell, The Katmai Group, Veniaminof) directly in the side panel alongside the interactive map. This feature bridges the gap between raw scientific data and immediate, actionable insights.

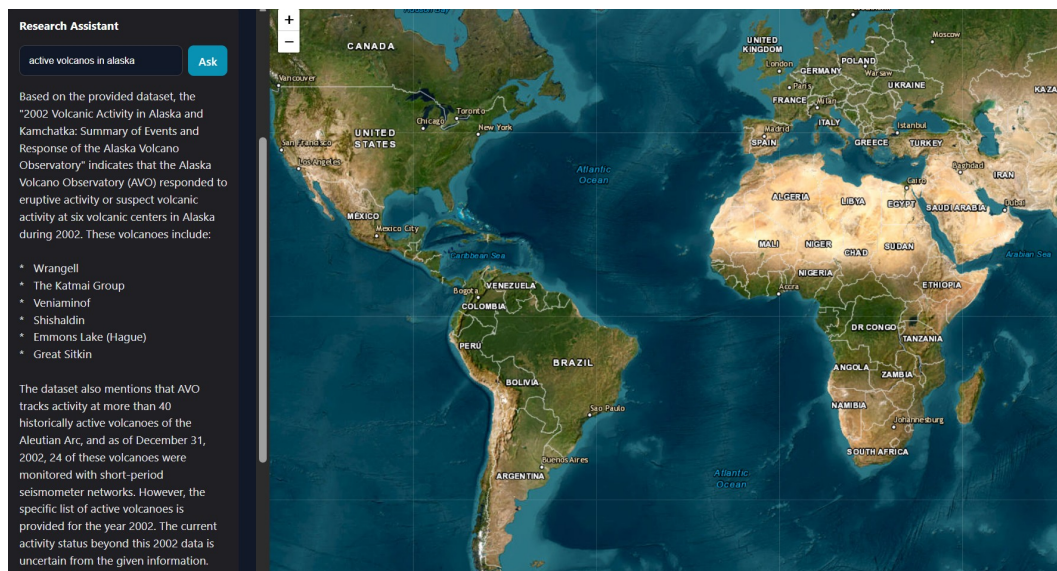


Fig. 6.5 The Research Assistant extracting specific volcano data from a dataset based on a user query

6.2.7 Results

The results indicate that integrating AI with geospatial data systems significantly improves accessibility and usability. The project demonstrates that natural-language interfaces combined with intelligent summarization can simplify complex scientific data exploration. The modular architecture also ensures flexibility for future improvements.

6.2.8 Limitations of the Results

The presented results should be interpreted in the context of current system limitations:

- dataset location mapping is approximate due to limited granule precision,
- performance depends on external API availability and response time,
- AI-generated summaries may occasionally lack domain-specific depth,
- query understanding is limited to basic parsing and keyword extraction.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

GeoScope was developed to address the difficulty of finding NASA datasets by treating Earth data discovery as a natural-language search problem. Instead of forcing users to manually navigate complex metadata catalogs, the system translates simple, intuitive queries (e.g., “*flood in India 2019*”) directly into meaningful dataset results.

The project successfully integrates a functional full-stack prototype. A Flask backend parses queries and searches NASA CMR collections, Google Gemini assists with keyword extraction and summarization, and a React/Vite frontend provides an interactive, map-based exploration interface. Together, these components deliver a clear end-to-end flow from natural-language input to dataset inspection, presenting summary bullets and spatial markers to make the data visually understandable.

Although the current prototype has limitations—such as reliance on Gemini for certain backend functions, occasional spatial inconsistencies, and leftover legacy code—it provides a strong proof of concept. In conclusion, GeoScope demonstrates that natural-language processing, scientific APIs, and modern map-based interfaces can be combined effectively to make Earth data search far more practical and accessible for researchers, students, and domain users.

7.2 Future Scope

GeoScope already provides a functional foundation for natural-language Earth data discovery, but several enhancements can improve its accuracy, reliability, and overall usefulness.

Advanced Search Intelligence and Ranking Future versions can improve natural-language understanding to better detect intent, event types, and locations for more precise matches. This enhanced parsing can be paired with a more advanced ranking layer that scores results based on spatial fit, temporal fit, metadata quality, and source reliability. Furthermore, introducing a user feedback mechanism to rate dataset relevance will allow the system to learn from real search behavior and continuously improve recommendations.

Spatial Accuracy and Coordinate Validation One important enhancement is to improve how bounding boxes, polygons, and marker coordinates are handled. A future version can validate NASA CMR spatial metadata more carefully, reduce coordinate-order ambiguity, and avoid synthetic fallback locations so the map reflects only real dataset geography.

More Informative Dataset Summaries The AI summary feature can be expanded to generate clearer explanations, citation-style references, and domain-specific insights. Future versions could also include short dataset comparisons, coverage notes, and confidence indicators to help users interpret results more effectively.

Additional Earth Data Sources The system can be extended beyond NASA CMR collections by integrating other Earth observation and geospatial sources. This may include climate datasets, disaster monitoring feeds, land use repositories, ocean data, and other scientific catalogs to broaden the search coverage.

Dynamic Data Updates At present, the app performs search on demand. A future version can support periodic refresh, improved caching, and live updates for rapidly changing event-related datasets. This would make the platform more useful for time-sensitive monitoring.

Personalized Discovery Different users may search for different purposes, such as research, policy, or disaster response. Future versions can maintain user preferences, saved searches, and dataset history so that recommendations become more personalized over time.

Cloud Deployment and Production Hardening The current architecture can be improved with stronger deployment support, centralized configuration, secure key management, and better handling of external API failures. This would make the application more stable and easier to deploy for real users.

References

- [1] A. Chapman *et al.*, “Dataset search: A survey,” *IEEE Intelligent Systems*, 2019.
- [2] R. Gomez, “Improving dense retrieval models with llm augmented data for dataset search,” *Knowledge-Based Systems*, 2024.
- [3] M. Wilson, “Context-aware search for environmental data using dense retrieval,” *ISPRS International Journal of Geo-Information*, 2024.
- [4] S. Lee and Y. Chen, “Keywords are not always the key: A metadata field analysis for natural language search on open data portals,” *arXiv*, 2025.
- [5] NASA, “Application of a dataset-publication knowledge graph for improving earth science data search,” NASA Technical Reports Server (NTRS), Tech. Rep., 2022.
- [6] J. Smith and A. Doe, “Metadata-based data exploration with retrieval-augmented generation for large language models,” *arXiv*, 2024.
- [7] P. Kumar, “A question answering dataset for temporal-sensitive retrieval-augmented generation,” *Scientific Data*, 2025.

Appendices

Appendix A

Core Code Snippets

1. **retriever.py** – *Hybrid Retrieval Orchestration*

Coordinates dual-search by invoking lexical and vector engines.

```
def hybrid_retrieve(query, year=None, region=None, limit=10):
    lexical_limit = max(limit * 4, 25)
    vector_limit = max(limit * 4, 25)

    lexical_results = local_lexical_search.search(
        query, limit=lexical_limit)
    vector_results = local_vector_search.search(
        query, limit=vector_limit)

    combined = combine_ranked_results(
        lexical_results,
        vector_results,
        year=year,
        region=region,
    )
    return combined[:limit]
```

2. **ranker.py** – *Hybrid Scoring and Ranking*

Weighted scoring formula combining semantic and keyword relevance.

```
def combine_ranked_results(lexical_results, vector_results,
                           year=None, region=None):
    lexical_weight = HYBRID_BM25_WEIGHT
    vector_weight = HYBRID_VECTOR_WEIGHT

    max_lexical = max((r.get("lexicalScore", 0.0)
                       for r in lexical_results), default=0.0)

    score = (lexical_weight * lex_comp +
             vector_weight * vec_comp +
             metadata_weight * metadata_comp)

    ranked.sort(key=lambda item: item["score"], reverse=True)
    return ranked
```

3. **vector_search.py** – *Vector Embedding and Fallback*

Generates embeddings with a TF-IDF fallback mechanism.

```
def _encode_texts(self, texts):
    try:
        model = self._load_sentence_model()
        vectors = model.encode(texts,
                                normalize_embeddings=True,
                                show_progress_bar=False)
        return np.asarray(vectors, dtype="float32")
    except Exception as exc:
        self._encoder, vectors = self._build_fallback_encoder(
            texts)
        return self._normalize_embeddings(vectors)
```

4. **rag_service.py** – *RAG Answer Generation*

Integrates the pipeline with Google Gemini for grounded responses.

```
def assistant_answer(user_query, year=None, region=None,
                     top_k=RAG_TOP_K):
    datasets = hybrid_retrieve(user_query, year=year,
                                region=region, limit=top_k)
    prompt = render_prompt("research_assistant.txt",
                            user_query=user_query,
                            dataset_context=_dataset_context(datasets))
    response = generate_with_timeout(model, prompt)
    return {"answer": response.text.strip(),
            "datasets": datasets,
            "source": "local_hybrid_rag"}
```

5. **build_index.py** – *Index Build Pipeline*

Prepares the metadata SQLite DB and FAISS vector index.

```
def main():
    records = load_jsonl(args.input)
    local_lexical_search.load(force=True)
    vector_info = local_vector_search.build(force=args.force)
    manifest = {"recordCount": len(records),
                "vectorBackend": vector_info.get("backend")}
    INDEX_MANIFEST_PATH.write_text(json.dumps(manifest,
                                                indent=2))
```